🐢🐰 Slow & Fast Pointer

Complete Visual Guide - Master the Tortoise & Hare!

Detect Cycles, Find Middle, Solve Array Problems

📚 Table of Contents

1. Introduction - The Pattern
2. Detect Cycle in Linked List
3. Find Cycle Start
4. Find Middle of List
5. Find Duplicate in Array
6. Happy Number
7. Summary & Patterns

1 The Slow & Fast Pointer Pattern

🐢 Tortoise & 🐰 Hare Algorithm

Also known as **Floyd's Cycle Detection Algorithm**

Uses **two pointers** moving at **different speeds**

🎯 Core Concept

Slow Pointer: Moves 1 step at a time (🐢 tortoise)
Fast Pointer: Moves 2 steps at a time (🐰 hare)

Key Insight: If there's a cycle, fast will eventually catch up to slow!

📊 ENHANCED Visual Example - Clear Initial Positions

```
🔍 STEP-BY-STEP: No Cycle Example

Linked List: 1 → 2 → 3 → 4 → 5 → null

╔════════════════════════════════════╗
║ STEP 0: INITIALIZATION             ║
╚════════════════════════════════════╝
BOTH pointers start at the FIRST NODE (head = 1)

  🐢 slow
  🐰 fast
   ↓  ↓
  [1] → [2] → [3] → [4] → [5] → null
   ↑
  HEAD (both start HERE!)


╔════════════════════════════════════╗
║ STEP 1: First Movement             ║
╚════════════════════════════════════╝
• slow moves 1 step: 1 → 2
• fast moves 2 steps: 1 → 2 → 3

         🐢          🐰
         ↓           ↓
  [1] → [2] → [3] → [4] → [5] → null


╔════════════════════════════════════╗
║ STEP 2: Second Movement            ║
╚════════════════════════════════════╝
• slow moves 1 step: 2 → 3
• fast moves 2 steps: 3 → 4 → 5

               🐢                🐰
               ↓                 ↓
  [1] → [2] → [3] → [4] → [5] → null


╔════════════════════════════════════╗
║ STEP 3: Fast Reaches End           ║
╚════════════════════════════════════╝
• slow moves 1 step: 3 → 4
• fast moves 2 steps: 5 → null → (stops!)

                     🐢
                     ↓
  [1] → [2] → [3] → [4] → [5] → null
                                 ↑
                                 🐰 fast = null

✅ RESULT: NO CYCLE DETECTED!
Fast pointer reached null → List has an end → No cycle
```

📊 WITH CYCLE - Clear Visualization

```
🔍 STEP-BY-STEP: With Cycle Example

⚠ CYCLE INFORMATION:
• Cycle exists: YES
• Cycle entry point: Node 2 (@200)
• Node 5's next pointer: Points back to @200 (creates the loop)
• Cycle path: 2 → 3 → 4 → 5 → 2 → 3 → 4 → 5 → ...

Linked List with Cycle:
       ┌─────────────┐
       │             ↓
  [1] → [2] → [3] → [4] → [5]
  @100  @200  @300  @400  @500
         ↑ ENTRY         │
         │               │
         └───────────────┘
          (Node 5 points back here!)

╔════════════════════════════════════╗
║ STEP 0: INITIALIZATION             ║
╚════════════════════════════════════╝
BOTH start at node 1

  🐢 slow = @100 (node 1)
  🐰 fast = @100 (node 1)
   ↓  ↓
  [1] → [2] → [3] → [4] → [5] ┐
  @100  @200  @300  @400  @500 │
         ↑_____________________│

Movement this step:
  • slow: START → @100
  • fast: START → @100


╔════════════════════════════════════╗
║ STEP 1: First Move                 ║
╚════════════════════════════════════╝
         🐢          🐰
         ↓           ↓
  [1] → [2] → [3] → [4] → [5] ┐
  @100  @200  @300  @400  @500 │
         ↑_____________________│

Movement this step:
  • slow: @100 → @200 (moved 1 step)
  • fast: @100 → @200 → @300 (moved 2 steps)

Current positions:
  🐢 slow = @200 (node 2)
  🐰 fast = @300 (node 3)


╔════════════════════════════════════╗
║ STEP 2: Second Move                ║
╚════════════════════════════════════╝
               🐢                🐰
               ↓                 ↓
  [1] → [2] → [3] → [4] → [5] ┐
  @100  @200  @300  @400  @500 │
         ↑_____________________│

Movement this step:
  • slow: @200 → @300 (moved 1 step)
  • fast: @300 → @400 → @500 (moved 2 steps)

Current positions:
  🐢 slow = @300 (node 3)
  🐰 fast = @500 (node 5)


╔════════════════════════════════════╗
║ STEP 3: Loop Back! (fast enters cycle) ║
╚════════════════════════════════════╝
                     🐢     🐰
                     ↓      ↓
  [1] → [2] → [3] → [4] → [5] ┐
  @100  @200  @300  @400  @500 │
         ↑_____________________│

Movement this step:
  • slow: @300 → @400 (moved 1 step)
  • fast: @500 → @200 → @300 (moved 2 steps, LOOPED BACK!)
         ↑ node 5's next points to @200
         ↑ then followed next to @300

Current positions:
  🐢 slow = @400 (node 4)
  🐰 fast = @300 (node 3) ← FAST WRAPPED AROUND!


╔════════════════════════════════════╗
║ STEP 4: COLLISION!                 ║
╚════════════════════════════════════╝
                          🐢🐰
                           ↓
  [1] → [2] → [3] → [4] → [5] ┐
  @100  @200  @300  @400  @500 │
         ↑_____________________│

Movement this step:
  • slow: @400 → @500 (moved 1 step)
  • fast: @300 → @400 → @500 (moved 2 steps)

Final positions:
  🐢 slow = @500 (node 5)
  🐰 fast = @500 (node 5)

🎉 COLLISION! slow == fast → CYCLE DETECTED!
Both pointers met at @500 (node 5)!

📊 Summary of all movements:
Step 0: slow=@100, fast=@100
Step 1: slow=@200, fast=@300
Step 2: slow=@300, fast=@500
Step 3: slow=@400, fast=@300 (fast looped!)
Step 4: slow=@500, fast=@500 ✓ MATCH!
```

✅ When to Use This Pattern

- ✓ **Detect cycles** in linked lists
- ✓ **Find middle** of linked list
- ✓ **Find cycle start** point
- ✓ **Detect duplicates** in arrays (Floyd's)
- ✓ **Happy number** problem
- ✓ Any problem involving **repeated sequences**

⚠ Space Complexity Advantage

Without slow/fast: Need HashSet → O(N) space
With slow/fast: Only two pointers → O(1) space!

2 Problem 1: Detect Cycle in Linked List

Problem: Determine if a linked list has a cycle
LeetCode: #141 - Linked List Cycle

🔍 How the Algorithm Works

Step-by-Step Breakdown:

1. INITIALIZATION:
- Both pointers start at the HEAD
- `slow = head` and `fast = head`

2. MOVEMENT RULES:
- **SLOW:** Moves 1 step per iteration → `slow = slow.next`
- **FAST:** Moves 2 steps per iteration → `fast = fast.next.next`

3. TERMINATION CONDITIONS:

✓ Scenario A: CYCLE EXISTS
- Fast will eventually catch up to slow
- When `slow == fast` → CYCLE DETECTED!
- Why? In a circular track, the faster runner MUST lap the slower runner

✗ Scenario B: NO CYCLE
- Fast will reach the end of the list
- When `fast == null` or `fast.next == null` → NO CYCLE
- Why? If there's an end, there's no loop!

4. WHY IT WORKS:
- **Gap closes by 1 each step:**
  → If slow is N nodes behind fast in the cycle
  → After 1 iteration: slow moves +1, fast moves +2
  → Gap decreases from N to N-1
  → Eventually gap = 0 → they meet!

- **Mathematical guarantee:**
  → In a cycle of length C
  → Gap closes by 1 per step
  → Maximum C steps to collision
  → Time complexity: O(N)

🎯 The Logic

Think of a race track:
- If track is straight (no cycle), fast finishes first
- If track is circular (has cycle), fast laps slow eventually

Key Insight: In a cycle, faster runner MUST catch slower runner!

📊 Cycle Example

```
⚠ CYCLE INFO: Node 5's next pointer goes back to Node 2

List with Cycle:
      ┌─────────────┐
      │             ↓
[1] → [2] → [3] → [4] → [5]
      ↑ ENTRY       │
      │             │
      └─────────────┘
       (Node 5 loops back to Node 2)

Step 0: slow=[1], fast=[1]
Step 1: slow=[2], fast=[3]
Step 2: slow=[3], fast=[5]
Step 3: slow=[4], fast=[3]  (fast wrapped around!)
Step 4: slow=[5], fast=[5]  ✓ THEY MEET! → Cycle detected!

If slow == fast → Cycle exists!
If fast reaches null → No cycle!
```

💻 Java Code (Fully Commented)

```java
/**
 * Detect Cycle in Linked List
 *
 * LeetCode 141 - Linked List Cycle
 *
 * Approach: Slow & Fast Pointer (Floyd's Algorithm)
 * - Slow moves 1 step
 * - Fast moves 2 steps
 * - If they meet → cycle exists
 * - If fast reaches null → no cycle
 *
 * Time: O(N) - worst case visit all nodes
 * Space: O(1) - only two pointers
 */
public boolean hasCycle(ListNode head) {
    // Edge case: empty list or single node
    if (head == null || head.next == null) {
        return false;
    }

    // Initialize both pointers at head
    ListNode slow = head;
    ListNode fast = head;

    // Move pointers until fast reaches end
    while (fast != null && fast.next != null) {
        // Slow moves 1 step (tortoise)
        slow = slow.next;

        // Fast moves 2 steps (hare)
        fast = fast.next.next;

        // If they meet → cycle detected!
        if (slow == fast) {
            return true;
        }
    }

    // Fast reached null → no cycle
    return false;
}

/*
 * Why this works mathematically:
 *
 * If cycle exists:
 * - Slow enters cycle at some point
 * - Fast is already in cycle (or enters after)
 * - Fast gains 1 step per iteration
 * - Eventually fast.position == slow.position
 *
 * Distance covered when they meet:
 * - Slow: x + y (x = distance to cycle, y = distance in cycle)
 * - Fast: x + y + nC (C = cycle length, n = laps)
 * - Since fast = 2×slow: x + y + nC = 2(x + y)
 * - Simplifies to: nC = x + y
 * - This proves they MUST meet in cycle!
 */
```

⚠ Common Mistakes

- ❌ Not checking `fast.next != null` → NullPointerException
- ❌ Starting pointers at different positions
- ❌ Moving fast only 1 step (becomes slow pointer!)

3 Problem 2: Find Cycle Start

Problem: Find WHERE the cycle begins
LeetCode: #142 - Linked List Cycle II

🔍 How the Algorithm Works

The Two-Phase Approach:

PHASE 1: DETECT THE CYCLE
- Use slow & fast pointers (same as Problem 1)
- When they meet → cycle exists, remember meeting point
- If fast reaches null → no cycle, return null

PHASE 2: FIND THE ENTRY POINT
- **Reset SLOW to HEAD**
- **Keep FAST at meeting point**
- Move **BOTH at same speed (1 step each)**
- Where they meet next = **CYCLE START!**

🤔 WHY DOES THIS WORK?

Let's use variables:
- `L1` = distance from HEAD to cycle ENTRY
- `L2` = distance from ENTRY to first MEETING point
- `C` = cycle length

When they first meet (Phase 1):
- SLOW traveled: `L1 + L2`
- FAST traveled: `L1 + L2 + nC` (n = number of full laps fast made)
- Since FAST moves 2× faster: `2(L1 + L2) = L1 + L2 + nC`

Simplify the equation:

```
2(L1 + L2) = L1 + L2 + nC
2L1 + 2L2 = L1 + L2 + nC
L1 + L2 = nC
L1 = nC - L2
```

🎯 KEY INSIGHT:
`L1 = nC - L2` means:
- Distance from **HEAD to ENTRY** = L1
- Distance from **MEETING to ENTRY** (going around cycle) = nC - L2
- **THESE ARE EQUAL!**

So if we:
1. Start SLOW at HEAD
2. Start FAST at MEETING point
3. Move both 1 step at a time

They will both reach the ENTRY point at the same time!

🎯 The Algorithm (Two Phases)

Phase 1: Detect cycle (same as before)
Phase 2: Find start point

Magic Step: Move one pointer to head, keep other at meeting point.
Move both 1 step at a time → they meet at cycle start!

📊 Visual Proof

```
⚠ CYCLE INFO: Node 5's next pointer goes back to Node 2 (cycle entry)

Phase 1: Detect cycle
      ┌─────────────┐
      │             ↓
[1] → [2] → [3] → [4] → [5]
      ↑ ENTRY       │
      │             │
      └─────────────┘
       (Cycle: 2→3→4→5→2→3→...)

slow & fast meet at node 5

Phase 2: Find start
slow = head = [1]
fast = [5] (meeting point)

Move both 1 step at a time:
slow=[1]→[2], fast=[5]→[2]  ✓ MEET AT [2]!

[2] is the cycle start!

Mathematical Proof:
Let x = distance from head to cycle start
Let y = distance from cycle start to meeting point
Let C = cycle length

When they meet:
- slow traveled: x + y
- fast traveled: x + y + nC (n laps)
- fast = 2×slow: x + y + nC = 2(x + y)
- Simplify: nC = x + y
- Therefore: x = nC - y

So distance from head to start = distance from meeting to start!
```

💻 Java Code (Fully Commented)

```java
/**
 * Find Cycle Start in Linked List
 *
 * LeetCode 142 - Linked List Cycle II
 *
 * Time: O(N)
 * Space: O(1)
 */
public ListNode detectCycle(ListNode head) {
    if (head == null || head.next == null) {
        return null;
    }

    // PHASE 1: Detect if cycle exists
    ListNode slow = head;
    ListNode fast = head;
    boolean hasCycle = false;

    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;

        if (slow == fast) {
            hasCycle = true;
            break;  // Found cycle, stop Phase 1
        }
    }

    // No cycle found
    if (!hasCycle) {
        return null;
    }

    // PHASE 2: Find cycle start
    // Move slow to head, keep fast at meeting point
    slow = head;

    // Move both 1 step at a time until they meet
    while (slow != fast) {
        slow = slow.next;    // 1 step
        fast = fast.next;    // 1 step (not 2!)
    }

    // Meeting point is cycle start
    return slow;  // or fast, both point to same node
}
```

4 Problem 3: Find Middle of Linked List

Problem: Find the middle node of a linked list
LeetCode: #876 - Middle of the Linked List

🔍 How the Algorithm Works

The Marathon Runner Analogy:

CONCEPT:
Imagine two runners in a marathon:
- Runner A (SLOW) runs at normal speed
- Runner B (FAST) runs at 2× speed
- When Runner B finishes, Runner A is at the halfway point!

ALGORITHM STEPS:

1. INITIALIZATION:
- Both start at HEAD: `slow = head, fast = head`

2. MOVEMENT:
- SLOW moves 1 step: `slow = slow.next`
- FAST moves 2 steps: `fast = fast.next.next`

3. TERMINATION:
- Stop when FAST reaches the end
- Conditions: `fast == null` OR `fast.next == null`

📐 WHY IT WORKS:

For ODD length list (e.g., 5 nodes):

```
List: [1] → [2] → [3] → [4] → [5]

Step 0: slow@1, fast@1
Step 1: slow@2, fast@3 (fast moved 2 steps)
Step 2: slow@3, fast@5 (fast at end!)

STOP! slow is at position 3 (middle of 5)
```

For EVEN length list (e.g., 4 nodes):

```
List: [1] → [2] → [3] → [4] → NULL

Step 0: slow@1, fast@1
Step 1: slow@2, fast@3
Step 2: slow@3, fast@NULL (fast.next would be null)

STOP! slow is at position 3 (second middle of 4)
```

🎯 MATHEMATICAL PROOF:
- Total nodes = N
- When FAST travels N steps (or N-1), SLOW travels N/2 steps
- N/2 is the middle position!

📝 NOTE ON EVEN LENGTH:
- For even-length lists, there are 2 middle nodes
- This algorithm returns the **second middle**
- Example: [1,2,3,4] → returns node 3 (not node 2)

🎯 The Logic

When fast reaches end, slow is at middle!

Why? Fast moves 2× faster than slow
When fast travels N steps, slow travels N/2 steps
N/2 is the middle!

📊 Visual Example

```
List: [1] → [2] → [3] → [4] → [5]

Step 0: slow=[1], fast=[1]
Step 1: slow=[2], fast=[3]
Step 2: slow=[3], fast=[5]
Step 3: fast.next=null, STOP

slow is at [3] → Middle node! ✓

For even length [1,2,3,4]:
Step 0: slow=[1], fast=[1]
Step 1: slow=[2], fast=[3]
Step 2: slow=[3], fast=null, STOP

slow is at [3] → Second middle! ✓
```

💻 Java Code

```java
/**
 * Find Middle of Linked List
 * LeetCode 876
 * Time: O(N), Space: O(1)
 */
public ListNode middleNode(ListNode head) {
    ListNode slow = head;
    ListNode fast = head;

    // When fast reaches end, slow is at middle
    while (fast != null && fast.next != null) {
        slow = slow.next;      // 1 step
        fast = fast.next.next; // 2 steps
    }

    return slow;  // Middle node
}
```

5 Problem 4: Find Duplicate in Array

Problem: Find duplicate in array [1..n] with values 1 to n
LeetCode: #287 - Find the Duplicate Number
Constraint: Can't modify array, O(1) space

🎯 Key Insight: Array IS a Linked List!

Treat array indices as nodes
Treat array values as "next" pointers

Example: arr = [1,3,4,2,2]
Index 0 → value 1 → "points to" index 1
Index 1 → value 3 → "points to" index 3
Duplicate creates a CYCLE!

📊 How Array Forms a Cycle

```
Array: [1, 3, 4, 2, 2]
Index:  0  1  2  3  4

Think of it as:
Index 0: value=1 → "next" is index 1
Index 1: value=3 → "next" is index 3
Index 2: value=4 → "next" is index 4
Index 3: value=2 → "next" is index 2
Index 4: value=2 → "next" is index 2

Path: 0 → 1 → 3 → 2 → 4 → 2 → 4 → 2...
                  ┌───┘
                  └ Cycle! Entry point = 2

The duplicate number is 2!
```

💻 Java Code

```java
/**
 * Find Duplicate in Array using Floyd's Algorithm
 * LeetCode 287
 * Time: O(N), Space: O(1)
 */
public int findDuplicate(int[] nums) {
    // Phase 1: Detect cycle
    int slow = nums[0];
    int fast = nums[0];

    do {
        slow = nums[slow];          // 1 step
        fast = nums[nums[fast]];    // 2 steps
    } while (slow != fast);

    // Phase 2: Find cycle start (the duplicate)
    slow = nums[0];

    while (slow != fast) {
        slow = nums[slow];
        fast = nums[fast];
    }

    return slow;  // The duplicate number
}
```

6 Problem 5: Happy Number

Problem: Determine if a number is "happy"
LeetCode: #202 - Happy Number

📖 Definition

A happy number is defined by:
1. Replace number with sum of squares of its digits
2. Repeat until number = 1 (happy!) or loops forever (sad)

Example: 19
$1^2 + 9^2 = 82$
$8^2 + 2^2 = 68$
$6^2 + 8^2 = 100$
$1^2 + 0^2 + 0^2 = 1$ ✓ Happy!

🎯 Why Slow & Fast?

If not happy → sequence enters a CYCLE
Use slow/fast to detect the cycle!
If slow == fast before reaching 1 → Not happy

💻 Java Code

```java
/**
 * Happy Number using Floyd's Algorithm
 * LeetCode 202
 * Time: O(log N), Space: O(1)
 */
public boolean isHappy(int n) {
    int slow = n;
    int fast = n;

    do {
        slow = sumSquares(slow);                // 1 step
        fast = sumSquares(sumSquares(fast));    // 2 steps

        if (slow == 1 || fast == 1) {
            return true;  // Happy!
        }
    } while (slow != fast);

    return false;  // Cycle detected, not happy
}

private int sumSquares(int n) {
    int sum = 0;
    while (n > 0) {
        int digit = n % 10;
        sum += digit * digit;
        n /= 10;
    }
    return sum;
}
```

7 Summary & Patterns

📊 Problem Patterns

Problem Type	Pattern	Key Points
Detect Cycle	slow=1, fast=2	If meet → cycle
Find Cycle Start	Two phases	Phase 2: both move 1 step
Find Middle	slow=1, fast=2	When fast ends, slow=middle
Array Duplicate	Treat as linked list	Index→value is "next"
Happy Number	Detect cycle in sequence	If cycle before 1 → not happy

✅ Common Template

```
// Initialize
slow = head
fast = head

// Phase 1: Detect
while (fast != null && fast.next != null) {
    slow = slow.next       // 1 step
    fast = fast.next.next  // 2 steps

    if (slow == fast) {
        // Cycle detected!
        break
    }
}

// Phase 2: Find start (if needed)
slow = head
while (slow != fast) {
    slow = slow.next  // Both 1 step
    fast = fast.next
}
```

⚡ Time & Space Complexity

Time: O(N) for all problems
Space: O(1) - only two pointers!

Alternative (HashSet):
Time: O(N), Space: O(N) ❌

Slow & Fast wins: Same time, better space! ✓

🎉 You've Mastered Slow & Fast Pointer!

You now understand:
- Floyd's Cycle Detection
- Finding cycle start
- Finding middle node
- Array as linked list trick
- O(1) space optimization

Practice these 5 problems to solidify!